Problem 1: A Small World (Knowledge Representation)

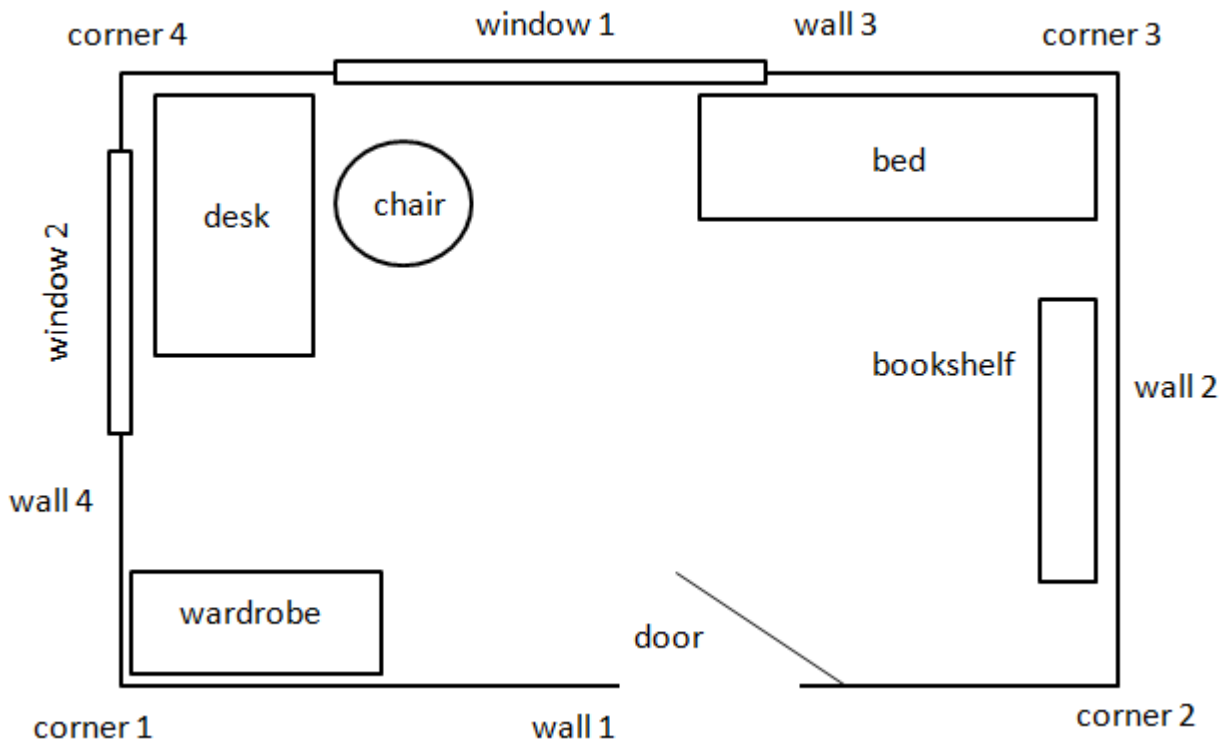


Fig. 1.1: A Student's Room

Represent this student's room such that the following queries are possible:

1. Which furniture is in the room?
2. How many doors, windows, tables, ... are in the room?
3. Where is the table, the chair, ... ?
4. What is to the left (right) of the table, ... (with respect to the center of the room)?
5. What is at the wall 2, at the window 1, ... ?
6. What is in the corner 1, ... ?

Hint:

Start with the reprentation of the queries and adapt the reprentation of the room correspondigly. Aim at a concise program with few facts and many rules.

```
furniture( wardrobe, 1).
furniture( bookshelf, 1).
furniture( bed, 1).
furniture( desk, 1).
```

```
furniture( chair, 1).

furniture( X) :- furniture( X, N).    /* Query 1: ?- furniture( X) */

number( X, N) :- furniture( X, N).  /* Query 2: ?- number( Object, Number)
*/
number( door, 1).
number( window, 2).

at( wall_1, wardrobe).               /* Query 5: ?- at( At, Object) */
at( wall_1, door).
at( wall_2, bookshelf).
at( wall_2, bed).
at( wall_3, bed).
at( wall_3, window_1).
at( wall_3, desk).
at( wall_4, desk).
at( wall_4, window_2).
at( wall_4, wardrobe).
at( window_1, bed).
at( window_2, desk).

in( corner_1, wardrobe).            /* Query 6: ?- in( Place, Object) */
in( corner_3, bed).
in( corner_4, desk).

place( X, P) :- at( P, X).         /* Query 3: ?- place( Object, Place) */
place( X, P) :- in( P, X).
place( chair, at_desk).

left( door, bookshelf).             /* Query 4: ?- left( Object1, Object2)
*/
left( bookshelf, bed).
left( bed, desk).
left( desk, wardrobe).
left( wardrobe, door).

right( X, Y) :- left( Y, X).     /* Query 4: ?- right( Object1, Object2) */
```

Problem 2: **The 4-Color Problem**

Find an "admissible" coloring of a map such that all adjacent countries have different colors. It had been conjectured for a long time that such a coloring is possible with only 4 colors. Write a Prolog program that finds all admissible colorings of a given map using the colors red, blue, yellow and green.

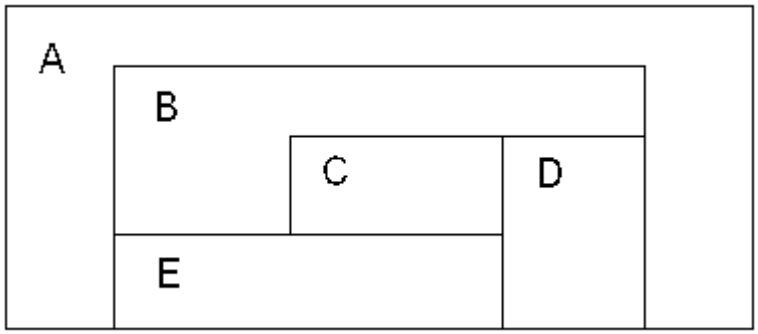


Fig. 2.1: A Map

Hint: The map can be specified by the following rules:

```
map( A, B, C, D, E) :-
    adjacent( A, B), adjacent( A, D), adjacent( A, E),
    adjacent( B, C), adjacent( B, D), adjacent( B, E),
    adjacent( C, D), adjacent( C, E),
    adjacent( D, E).

color( red).
color( bue).
color( yellow).
color( green).

adjacent( X, Y) :- color( X), color (Y), X \= Y.
```

Problem 3: **Europe**

Write a database with European countries specifying the area, the population and the neighbors of the individual countries:

```
area( austria, 83858).
area( france, 547030).
area( germany, 357021).
area( italy, 301230).
area( liechtenstein, 160).
area( spain, 504851).
area( switzerland, 41290).
area( united_kingdom, 244820).
. . .
population( austria, 8169929).
poulation( france, 63182000).
poulation( germany, 83251851).
poulation( italy, 59530464).
poulation( liechtenstein, 32842).
poulation( spain, 47059533).
poulation( switzerland, 7507000).
poulation( united_kingdom, 61100835).
. . .
neighbor( austria, switzerland).
neighbor( france, switzerland).
neighbor( france, germany).
```

```
neighbor( france, spain).
neighbor( germany, switzerland).
neighbor( italy, switzerland).
neighbor( liechtenstein, switzerland).
. . .
```

Enhance the database with deduction rules, e.g.

```
density( S, D) /* D ist die population density of the state S */
```

such that the following queries are possible:

1. Which state is a neighbor of Switzerland?
2. Which state is a neighbor of France?
3. Is there a neighbor of Germany that has a bigger population and a larger area than Germany?
4. Is there a neighbor of Switzerland that has a higher density than Switzerland?

```
density( S, D) :- population( S, E), area( S, F), D is E/F.

neighbors( S1, S2) :- neighbor( S1, S2).
neighbors( S1, S2) :- neighbor( S2, S1).

neighbor_ch( S) :- neighbors( ch, S).          /* Query 1 */

neighbor_f( S) :- neighbors( f, S).            /* Query 2 */

neighbor_d( S) :-                              /* Query 3 */
    neighbors( d, S),
    population( d, ED), population( S, ES),
    area( d, FD), area( S, FS),
    ES > ED, FS > FD.

neighbor_ch1( S) :-                            /* Query 4 */
    neighbors( ch, S),
    density( ch, DCH), density( S, DS), DS > DCH.
```

Reference :

1. Website : https://www.nosco.ch/ai/en/exercise_04.php#4.4